



Appendix

Chapter 1, Artificial Neural Network Fundamentals

1. What are the various layers in a neural network?

Input, hidden, and output.

2. What is the output of feedforward propagation?

Predictions that help in calculating the loss value.

3. How is the loss function of a continuous dependent variable different from that of a binary dependent variable or a categorical dependent variable?

Mean squared error (MSE) is the generally used loss function for continuous dependent variables, and binary cross-entropy is generally used for binary dependent variables. Categorical cross-entropy is used for categorical dependent variables.

4. What is stochastic gradient descent?

It is the process of reducing loss by adjusting weights in the direction of decreasing gradient.

5. What does a backpropagation exercise do?

It computes the gradients of all weights with respect to loss using the chain rule.

6. How does the update of all the weights across layers happen during backpropagation?

It happens using the formula $\mathbf{W}_{\text{new}} = \mathbf{W} - \alpha * (\text{dL}/\text{dW})$.

7. What functions are performed within each epoch of training a neural network?

For each batch in an epoch, you perform forward-propagation, compute the loss, compute the weight gradients with backpropagation on the loss, and update the weights. You repeat with the next batch until all epochs are finished.

8. Why is training a network on a GPU faster when compared to training it on a CPU?

More matrix operations can be performed in parallel on GPU hardware.

9. What is the impact of the learning rate when training a neural network?

Too high a learning rate will explode the weights, and too small a learning rate will not change the weights at all.

10. What is the typical value of the learning rate parameter?

It's usually between $1\text{e-}2$ and $1\text{e-}5$, but it is dependent on many factors, such as the architecture, dataset, and optimizers being used.

Chapter 2, PyTorch Fundamentals

1. Why should we convert integer inputs into float values during training?

`nn.Linear` (and almost all torch layers) only accepts floats as inputs.

2. What are the methods used to reshape a tensor object?

`tensor.view(shape)`, `permute`, `flatten`, `squeeze`, and `unsqueeze`.

3. Why is computation faster with tensor objects than with NumPy arrays?

The capability to run on GPUs in parallel is only available on tensor objects.

4. What constitutes the `init` magic function in a neural network class?

Calling `super().__init__()` and specifying the neural network layers.

5. Why do we perform zero gradients before performing backpropagation?

To ensure that gradients from previous calculations are flushed out.

6. What magic functions constitute the dataset class?

`__len__` and `__getitem__`.

7. How do we make predictions on new data points?

By calling the model on the tensor as if it is a function – `model(x)`.

8. How do we fetch the intermediate layer values of a neural network?

By creating a custom method that can perform `forward` only up to intermediate layers, or by returning the intermediate layer values as an additional output in the forward method itself.

9. How does the `Sequential` method help simplify the definition of the architecture of a neural network?

We can avoid creating `__init__` and the `forward` method by connecting a sequence of layers.

10. While updating `loss_history`, we append `loss.item()` instead of `loss`. What does this accomplish, and why is it useful to append `loss.item()` instead of just `loss`?

`loss.item()` converts a 0-dimension torch tensor into a Python float that takes up less memory, is stored on the CPU, and can be used by other libraries, such as plotting, stats, etc.

11. What are the advantages of using `torch.save(model.state_dict())`?

By saving only the model's state dictionary rather than the entire model object, you can significantly reduce the storage space required. Also, this approach makes it easier to transfer models between different devices and environments, facilitating model deployment and sharing. Because a state dict is a dictionary, the model (and its intermediate layers) can be copied, updated, altered, and stored independently.

Chapter 3, Building a Deep Neural Network with PyTorch

1. What happens if input values are not scaled in the input dataset?

It takes longer to adjust weights to the optimal value because input values vary so widely when they are unscaled. There is a chance that the model might not learn at all.

2. What could be the issue if a background has a white pixel color while the content has a black pixel color when training a neural network?

The mean of the data is close to 1 (1 being white and 0 being black). This may cause the neural network to take longer to train as a lot of neurons get activated initially. The network needs to learn – in the beginning stages – to ignore a majority of the not-so-useful content that is white in color.

3. What is the impact of batch size on the model's training time and memory?

The larger the batch size, the greater the time and memory required to train the batch.

4. What is the impact of the input value range on weight distribution at the end of training?

If the input value is not scaled to a certain range, certain weights can result in overfitting or cause vanishing/exploding gradients.

5. How does batch normalization help improve accuracy?

Just as it is important that we scale inputs for better convergence of the ANN, batch normalization scales activations for better conver-

gence of its next layer.

6. Why do weights behave differently during training and evaluation in the dropout layer?

During training, weights are scaled down by dropout, which randomly sets a fraction of input units to zero, aiding in preventing overfitting. During evaluation, dropout is typically turned off, causing the weights to behave differently as the full network capacity is utilized without scaling down.

7. How do we know if a model has overfitted on training data?

The validation loss will be much higher than the training loss.

8. How does regularization help in avoiding overfitting?

Regularization techniques help the model to train in a constrained environment, thereby forcing the ANN to adjust its weights in a less biased fashion.

9. How do L1 and L2 regularization differ from each other?

L1 is the sum of the absolute value of weights, while L2 is the sum of the square of weights added to the loss value and the typical loss.

10. How does dropout help in reducing overfitting?

By dropping some connections in the ANN, we force individual neurons to learn from less data as well as from different subsets of inputs at every iteration. This forces the model to learn without relying on specific neurons or a fixed set of connections.

Chapter 4, Introducing Convolutional Neural Networks

1. Why was the prediction on the translated image in the first section of the chapter low when using traditional neural networks?

All images were centered in the original dataset, so the ANN only learned the task for centered images.

2. How is convolution done?

Convolution is performed by sliding a small filter or kernel over the input signal, multiplying the values element-wise, and summing them up to produce an output value at each position. This process is repeated for every position in the input signal, resulting in a new output signal that represents the presence of patterns or features within the original input signal

3. How are optimal weight values in a filter identified?

Through backpropagation.

4. How does the combination of convolution and pooling help in addressing the issue of image translation?

While convolution gives important image features, pooling takes the most prominent features in a patch of the image. This makes pooling a robust operation; even if something is translated using a few pixels, pooling will still return the expected output.

5. What do the convolution filters in layers close to the input learn?

Low-level features like edges.

6. What functionality does pooling do that helps in building a model?

It reduces the input size by reducing feature map size and makes model translation invariant.

7. Why can't we take an input image, flatten it just like we did on the Fashion-MNIST dataset, and train a model for real-world images?

If the image size is even modestly large, the number of parameters connecting two layers will be in the millions, resulting in considerable compute and potentially, unstable training.

8. How does data augmentation help in improving image translation?

Data augmentation creates copies of images that are translated by a few pixels. Therefore, the model is forced to learn the right classes even if the object in the image is off-center.

9. In what scenario do we leverage `collate_fn` for dataloaders?

When we need to perform batch-level transformations, which are difficult/slow to perform using `__getitem__`.

10. What impact does varying the number of training data points have on the classification accuracy of the validation dataset?

In general, the larger the dataset size, the better the model accuracy.

Chapter 5, Transfer Learning for Image Classification

1. What are VGG and ResNet pre-trained architectures trained on?

The images in the ImageNet dataset.

2. Why does VGG11 have inferior accuracy to VGG16?

VGG11 has fewer layers/blocks/parameters than VGG16.

3. What does the number 11 in VGG11 represent?

The 11 layer groups. Each group has two convolutional layers followed by ReLU and maxpool2d.

4. What does the term “residual” mean in “residual network”?

The term “residual” refers to the concept of residual learning, where there is a shortcut connection that skips one or more layers and directly adds the input to the output of a subsequent layer, helping in training very deep networks.

5. What is the advantage of a residual network?

It helps to prevent vanishing gradients and also helps to increase model depth without losing accuracy, by allowing gradients to directly pass through to the initial layers via shortcut connections.

6. What are the various popular pre-trained models discussed in the book and what is the speciality of each network?

AlexNet was the first successful convolutional neural network. VGG improved on AlexNet by making it deeper. Inception introduced an inception layer, which has multiple convolutional filters of different

sizes and pooling operations. ResNet introduced skip connections, helping to create even deeper networks.

7. During transfer learning, why should images be normalized with the same mean and standard deviation as those that were used during the training of the pre-trained model?

Models are trained such that they expect input images to be normalized with a specific mean and standard deviation.

8. When and why do we freeze certain parameters in a model?

We freeze certain parameters (typically called the backbone of the model) during retraining activities so that those parameters will not be updated during backpropagation. They need not be updated as they are already well trained, and this will also speed up the training time.

9. How do we know the various modules that are present in a pre-trained model?

```
print(model)
```

10. How do we train a model that predicts categorical and numerical values together?

By having multiple prediction heads and training with a separate loss for each head.

11. Why might age and gender prediction code not always work for an image of your own if we were to execute the same code as that which we wrote in the *Implementing age estimation and gender classification* section?

An image that does not have a similar distribution to the training data can give unexpected results. The image might come from a different demography/location.

12. How can we further improve the accuracy of the facial keypoint recognition model that we discussed in the *Implementing facial keypoint prediction* section?

We can add noise, color, and geometric augmentations to the training process.

Chapter 6, Practical Aspects of Image Classification

1. How are class activation maps obtained?

Refer to the eight steps provided in the *Generating CAMs* section.

2. How do batch normalization and data augmentation help when training a model?

They help reduce overfitting. Batch normalization helps stabilize the learning process by normalizing the incoming data at each layer, while data augmentation increases the diversity of the training set.

3. What are the common reasons why a CNN model overfits?

Too many CNN layers, no batch normalization, a lack of data augmentation, and a lack of dropout.

4. What are the various scenarios where a CNN model works with training and validation data at the data scientists' end but not in the real world?

Apart from obvious scenarios such as using different model/library versions than were used during training, real-world data can have a different distribution from the data used to train and validate a model due to several factors, such as environment shift, sensor shift, and so on. Additionally, the model might have overfitted on training data.

5. What are the various scenarios where we leverage OpenCV packages and when it is advantageous to use OpenCV over deep learning?

When working in constrained environments, where we know that the behavior of images has a limited scope, we prefer to use OpenCV as

the time to code solutions is much faster. It is also preferred when speed to infer is more important in constrained environments.

Chapter 7, Basics of Object Detection

1. How does the region proposal technique generate proposals?

It identifies regions that are similar in color, texture, size, and shape.

2. How is Intersection Over Union (IoU) calculated if there are multiple objects in an image?

IoU is independently calculated for each object with the ground truth, using the IoU metric, which is a quotient where the numerator is the intersection area between an object and the ground truth, and the denominator is the union area between an object and the ground truth.

3. Why is Fast R-CNN faster than R-CNN?

For all proposals, extracting the feature map from the VGG backbone is common. Reusing this feature map reduces almost 90% of the computations in Fast R-CNN as compared to R-CNN, which computes these features again and again for all proposals.

4. How does RoI pooling work?

All the cropped images coming from `selectivesearch` are passed through an adaptive max-pooling kernel so that the final output is of the same size.

5. What is the impact of not having multiple layers after obtaining a feature map when predicting bounding box corrections?

The model will struggle to capture complex relationships between features and fail to produce accurate bounding box corrections.

6. Why do we have to assign a higher weightage to regression loss when calculating overall loss?

Classification loss is cross-entropy, which is generally of the order $\log(n)$, resulting in outputs that can have a high range. However, bounding box regression losses are between 0 and 1. As such, regression losses have to be scaled up.

7. How does non-max suppression work?

By combining boxes of the same classes and with high IoUs, we eliminate redundant bounding box predictions.

Chapter 8, Advanced Object Detection

1. Why is Faster R-CNN faster when compared to Fast R-CNN?

We do not need to feed a lot of unnecessary proposals every time using the `selectivesearch` technique. Instead, Faster R-CNN automatically finds them using the region proposal network.

2. How are YOLO and SSD faster when compared to Faster R-CNN?

We don't need to rely on a new proposal network. The network directly finds the proposals in a single go.

3. What makes YOLO and SSD single-shot detector algorithms?

Networks predict all the proposals and predictions in one shot.

4. What is the difference between the objectness score and class score?

The objectness score identifies if an object exists or not, but the class score predicts the class for an anchor box whose objectness is non-zero.

Chapter 9, Image Segmentation

1. How does upscaling help in the U-Net architecture?

Upscaling helps the feature map to increase in size so that the final output is the same size as the input size.

2. Why do we need to have a fully convolutional network in U-Net?

Because both the inputs and outputs are images, it is difficult to predict an image-shaped tensor using the linear layer.

3. How does RoI Align improve upon RoI pooling in Mask R-CNN?

RoI Align takes offsets of predicted proposals to fine-align the feature map.

4. What is the major difference between U-Net and Mask R-CNN for segmentation?

U-Net is fully convolutional and has a single end2end network, whereas Mask R-CNN uses mini networks, such as Backbone, RPN, etc, to do different tasks. Mask R-CNN is capable of identifying and separating several objects of the same type, but U-Net can only identify (not separate them into individual instances).

5. What is instance segmentation?

If there are different objects of the same class in the same image, then each object is called an instance. Applying image segmentation to predict, at the pixel level, all the instances separately is called instance segmentation.

Chapter 10, Applications of Object Detection and Segmentation

1. Why is it important to convert datasets into a specific format for Detectron2?

Detectron2 is a framework that can train/evaluate multiple architectures at the same time. To achieve such flexibility with the same code, it is important to create certain restrictions on the datasets so that the framework can manipulate the datasets. That is why it is recommended that all Detectron2 datasets be in the COCO format.

2. It is hard to directly perform a regression of the number of people in an image. What is the key insight that allowed the VGG architecture to perform crowd counting?

We converted the target image into a heat map where the sum of intensity of all the pixels is equal to the number of people in the image.

3. Explain self-supervision in the case of image-colorization.

We were able to create input output pairs from the given images by simply converting images into black and white (BW). We treated the BW images as input and the color images as the intended output.

4. How did we convert a 3D point cloud into an image that is compatible with YOLO?

At every point in time, we viewed the 3D point cloud from top-down view and used the red channel to encode the highest point, green channel for intensity of the highest point, and blue channel for the density of points.

5. What is a simple way to handle videos using architectures that work only with images?

A simple way of doing this is to treat the frames' dimensions as a batch dimension and pool all the feature vectors of all the frames to get a single feature vector.

Chapter 11, Autoencoders and Image Manipulation

1. What is the “encoder” in “autoencoder”?

A neural network that converts an image into a vector representation.

2. What loss function does an autoencoder optimize for?

The pixel-level MSE, directly comparing the prediction with the input.

3. How do autoencoders help in grouping similar images?

Similar images will return similar encodings, which are easier to cluster.

4. When is a convolutional autoencoder useful?

When the inputs are images, using a convolutional autoencoder helps in tasks such as image denoising, image reconstruction, anomaly detection, image data drift, quality control, and so on.

5. Why do we get non-intuitive images if we randomly sample from the vector space of embeddings obtained from a vanilla/convolutional autoencoder?

The range of values in encodings is unconstrained, so proper outputs are highly dependent on the right range of values. Random sampling, in general, assumes a 0 mean and 1 standard deviation, which may not be the range of good encodings.

6. What are the loss functions that a variational autoencoder optimizes for?

The pixel-level MSE and the KL divergence of the distribution of the mean and standard deviation from the encoder.

7. How does the variational autoencoder overcome the limitation of vanilla/convolutional auto-encoders to generate new images?

By constraining predicted encodings to have a normal distribution, all encodings fall in the region of mean 0 and standard deviation 1, which is easy to sample from.

8. During an adversarial attack, why do we modify the input image pixels and not the weight values?

We do not have control over the neural network in adversarial attacks.

9. In a neural style transfer, what are the losses that we optimize for?

The perceptual (VGG) loss of the generated image with the original image and the style loss coming from the gram matrices of the generated and style images.

10. Why do we consider the activation of different layers and not the original image when calculating style and content loss?

Using more intermediate layers ensures that the generated image preserves finer details about the image. Also, using more losses makes the gradient ascent more stable.

11. Why do we consider the gram matrix loss and not the difference between images when calculating the style loss?

The gram matrix loss gives an indication of the style of the image, i.e., how the textures shapes, and colors are arranged, and will ignore the actual content. That is why it is more convenient for style loss.

12. Why do we warp images when building a model to generate deepfakes?

Warping images helps act as a regularizer. Further, it helps in generating as many images as required, helping in augmenting the dataset.

Chapter 12, Image Generation Using GANs

1. What happens if the learning rate of generator and discriminator models is high?

The model stability will be low.

2. In a scenario where the generator and discriminator are very well trained, what is the probability of a given image being real?

0.5

3. Why do we use ConvTranspose2d to generate images?

We cannot upscale/generate images using a linear layer.

ConvTranspose2d is a parametrized/neural-network-enabled way of upsampling images to a larger resolution.

4. Why do we have embeddings with a higher embedding size than the number of classes in conditional GANs?

Using more parameters gives the model more degrees of freedom to learn the important features of each class.

5. How can we generate images of men with beards?

By using a conditional GAN. Just as we had male and female images, we can have images of bearded males and other such classes while training our model.

6. Why do we have Tanh activation in the last layer in the generator and not ReLU or sigmoid?

The pixel range of normalized images is `[-1,1]`, and hence we use Tanh.

7. Why did we get realistic images even though we did not denormalize the generated data?

Even though the pixel values were not between `[0,255]`, the relative values were sufficient for the `make_grid` utility to de-normalize input.

8. What happens if we do not crop faces corresponding to images before training a GAN?

If there is too much background, the GAN can get wrong signals as to what is a face and what is not, so it might focus on generating more realistic backgrounds.

9. Why do the weights of the discriminator not get updated when the training generator is updated (as the `generator_train_step` function involves the discriminator network)?

It is a step-by-step process. When updating the generator, we assume the discriminator is able to do its best.

10. Why do we fetch losses on both real and fake images while training the discriminator but only the loss on fake images while training the generator?

Because whatever the generator creates are only fake images.

Chapter 13, Advanced GANs to Manipulate Images

1. Why do we need a Pix2Pix GAN if a supervised learning algorithm such as U-Net could have worked to generate images from contours?

U-Net only uses pixel-level loss during training. We needed Pix2Pix since there is no loss of realism when U-Net generates images.

2. Why do we need to optimize for three different loss functions in CycleGAN?

In CycleGAN, we optimize adversarial loss, cycle consistency loss, and identity loss in order to learn a more accurate mapping between two domains while preserving the original image's structure and appearance. A detailed answer is provided in the seven points of the *How CycleGAN works* section.

3. How do the tricks used by ProgressiveGAN help in building a StyleGAN model?

ProgressiveGAN helps the network to learn a few upsampling layers at a time so that when the image has to be increased in size, the networks responsible for generating current-sized images are optimal.

4. How do we identify the latent vectors that correspond to a given custom image?

By adjusting the randomly generated noise in such a way that the MSE loss between the generated image and the image of interest is as small as possible.

Chapter 14, Combining Computer Vision and Reinforcement Learning

1. How does an agent calculate the value of a given state?

By computing the expected reward at that state.

2. How is a Q-table populated?

By computing the expected reward for each state-action pair, which is the sum of the immediate reward and the expected future reward. This calculation is refined with each episode, thereby improving the estimates every time.

3. Why do we have a discount factor in a state action value calculation?

Due to uncertainty, we are unsure of how the future might work. Hence, we reduce future rewards' weightage, which is done by way of discounting.

4. Why do we need the exploration-exploitation strategy?

Only exploitation will make the model stagnant and predictable, and hence the model should be able to explore and find unseen steps that might be even more rewarding than what the model has already learned.

5. Why do we need to use Deep Q-Learning?

We let the neural network learn the likely reward system without the need for costly algorithms that may take too much time or demand visibility of the entire environment.

6. How is the value of a given state-action combination calculated using Deep Q-Learning?

It is simply the output of the neural network. The input is the state and the network predicts one expected reward for every action in the given state.

7. Once an agent has maximized a reward in the CartPole environment, is there a chance that it can learn a suboptimal policy later?

There is a small, non-zero chance of it learning something sub-optimal if the neural network experiences forgetting due to bad episodes.

Chapter 15, Combining Computer Vision and NLP Techniques

1. What are the inputs, steps for calculation, and outputs of self-attention?

The inputs to self-attention are a sequence of vectors. The steps involve computing attention scores between each pair of vectors by first converting the vectors into key vectors and query vectors. These two are matrix-multiplied, followed by a softmax function to obtain attention weights. The attention weights are then used to compute a weighted sum of the value vectors. The resulting context vectors are combined and typically passed through additional layers, such as feed-forward neural networks, to produce the final output.

2. How is an image transformed into a sequence input in a vision transformer?

In a vision transformer, an image is first cut into a fixed-size grid of patches, which are sent through a Conv2D layer to turn into sequences of vectors.

3. What are the inputs to the BERT transformer in a LayoutLM model?

The text contained in the box and the 2D position of the box are the inputs to the model.

4. What are the three objectives of BLIP?

Image-text matching, image-grounded text generation, and image-text contrastive learning.

Chapter 16, Foundation Models in Computer Vision

1. How are text and images represented in the same domain using CLIP?

By using contrastive loss, we force the embeddings from images and text from the same source to be as similar as possible while simultaneously ensuring that the embeddings of disparate sources are as different as possible.

2. How are different types of tokens, such as point tokens, bounding box tokens, and text tokens, calculated in the Segment Anything architecture?

By using a prompt-encoder model that accepts points, boxes, or text, and by using a pre-training task of segmenting a meaningful object underneath the point, within the box, or as described by the text.

3. How do diffusion models work?

They work by gradually denoising a noisy image from full noise to partial noise to no noise. At every step, the model ensures that the image generated is only slightly better than the current image.

4. What makes Stable Diffusion different from normal diffusion?

Unlike in normal diffusion, where the model works directly on images, in Stable Diffusion, the whole of the denoising training occurs in a latent space that is encoded/decodable by a variational autoencoder, making the training significantly faster.

5. What is the difference between Stable Diffusion and the SDXL model?

SDXL has been trained on images of size 1,024, whereas the standard model works in the 512-pixel space.

Chapter 17, Applications of Stable Diffusion

1. What is the key concept behind image in-painting using Stable Diffusion?

By generating latents only in the masked area and by preserving latents in the unmasked area, we ensure that we keep spatial consistency in the background while modifying the foreground to our desire.

2. What are the key concepts behind ControlNet?

There are two key concepts. First, we make a ControlNet branch by copying the downsampling path of the Stable Diffusion UNet2D model. All the modules in this branch have a zero-convolutional layer as a final layer and each module is added as a skip connection to the corresponding upsampling branch. Second, by training the new branch exclusively to accept special images such as canny, the training is faster.

3. What makes SDXL-Turbo faster than SDXL?

SDXL-Turbo follows a teacher-student training paradigm where the student learns to denoise several steps at once, making the student predict the same output as the teacher in fewer steps, i.e., faster.

4. What is the key concept behind DepthNet?

Modifying the Stable Diffusion model UNet2D to accept five channels instead of the usual four, with the fifth channel being a depth map. This allows the prediction of an image that is depth-map accurate.

Chapter 18, Moving a Model to Production

1. What is the REST API and what does it do?

It is an interface for programs to communicate over the internet using HTTP methods.

2. What is Docker and why is it important for deploying deep learning applications?

Docker is a containerization platform that allows developers to package, ship, and run applications in closed environments called containers. These play a handy role in deep learning deployments as the developer need not worry about downloading/installing libraries over multiple machines and can scale containers up/down depending on the load.

3. What is a simple and common technique for detecting unusual or novel images in a production setting that differ substantially from those used during training?

We first measure the distance between the new image's feature vectors and the feature vectors of training data. If this distance is too large compared to the distance of a sample image from validation data, we can tell if the image is unusual or not.

4. How can we speed up the similarity search of an image a large volume of vectors (millions)?

By using existing libraries such as FAISS. These methods typically pre-index large volumes of vectors, based on techniques such as clustering, for faster retrieval of likely vector candidates during the similarity search.

Learn more on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/modcv>

